IoT Challenge 2

Packet Sniffing Report

Marco Abbadessa - 10854816
Alice Berta - 10889590 - leader

Academic Year 2025–2026

Contents

1 Introduction **2**

2 Preliminary Network Analysis on `B.pcapng` **2**

3 Questions on `A.pcapng` **3**
3.1 CQ1) . . . 3
3.2 CQ2) . . . 3
3.3 CQ3) . . . 3
3.4 CQ4) . . . 3

4 Questions on `B.pcapng` **4**
4.1 CQ5) Last Will messages received through wildcard subscriptions . . . 4
4.2 CQ6a) Clients that erased a retained value on the public HiveMQ broker . . . 5
4.3 CQ6b) Of those clients, how many used a Client ID longer than 7 bytes? . . . 6
4.4 CQ7) Subscribe requests to the local broker with at least two wildcards . . . 6

5 Question on Both Capture Files **7**
5.1 CQ8) . . . 7

6 Final Answers Summary **7**

7 Conclusion **8**

1 Introduction

The goal of this challenge is to analyze two packet captures, namely `A.pcapng` and `B.pcapng`, in order to answer a set of protocol-level questions related to CoAP, MQTT, and MQTT-SN communications. The analysis was carried out mainly with Wireshark, using protocol-specific display filters and manual packet inspection. Whenever needed, additional fields were added as custom columns in order to simplify the identification of flows, brokers, topics, and client identifiers.

The challenge is divided into two main parts. The first part concerns the analysis of the packet captures and the derivation of the answers to CQ1–CQ8. The second part is a separate theoretical exercise about communication energy consumption in a smart building scenario. In this report we focus on the motivated answers for the packet analysis questions.

Before addressing the individual questions on `B.pcapng`, it was necessary to perform a preliminary network inspection in order to identify which brokers were present in the capture and how the MQTT traffic was distributed among them. This initial step was essential because the wording of several questions explicitly distinguishes between the local broker and the public HiveMQ broker. Therefore, the rest of the analysis relies on a precise identification of the network endpoints involved.

2 Preliminary Network Analysis on `B.pcapng`

As a first step, all MQTT `CONNECT` packets were isolated with the filter:

```
mqtt.msgtype == 1
```

By inspecting the destination IP addresses of the resulting packets, two clearly different traffic patterns emerged. On the one hand, some connections involved source and destination address `::1`, which is the IPv6 loopback interface. On the other hand, several connections originated from the host `10.0.2.15` and were directed to external public IP addresses.

The local broker was identified through the packets addressed to `::1`. This address is the IPv6 loopback address and is equivalent to `127.0.0.1` in IPv4. Its presence indicates that the broker and the client are running on the same machine, which is typical of a local or simulated setup. Moreover, starting from a certain point in the capture, the Client IDs used in the connections toward `::1` show the prefix `mosq-`, which is automatically generated by Mosquitto when clients do not explicitly provide an identifier. This strongly suggests that the local broker is a **Mosquitto** instance.

To identify the public HiveMQ broker, DNS traffic was inspected with the filter:

```
dns.qry.name contains "hivemq"
```

The DNS replies explicitly resolved `broker.hivemq.com` to three IPv4 addresses:

- `18.192.151.104`
- `35.158.43.69`
- `35.158.34.213`

These addresses correspond to AWS-hosted servers in the Frankfurt region. The identification was then confirmed by filtering the MQTT traffic involving those IPs:

```
mqtt and (ip.addr == 18.192.151.104 or ip.addr == 35.158.43.69 or ip.addr ==
    35.158.34.213)
```

This filter showed effective unencrypted MQTT traffic toward two of the three HiveMQ addresses, namely `18.192.151.104` and `35.158.43.69`. The address `35.158.34.213`, although returned by DNS as a valid HiveMQ endpoint, does not appear in any MQTT packet of the capture.

A fourth external IP address, namely `5.196.78.28`, is also present in the capture and carries MQTT traffic. However, since it is not associated with the HiveMQ DNS resolution and belongs to a different hosting provider, it must correspond to a different public broker.

In summary, the broker mapping used throughout the rest of the report is the following:

- local broker: `::1` (Mosquitto);
- public HiveMQ broker: `18.192.151.104` and `35.158.43.69`;
- other public broker: `5.196.78.28`.

This classification was used as the basis for all subsequent filters.

3 Questions on `A.pcapng`

3.1 CQ1)

TODO: Marco: add the complete analysis for CQ1a and CQ1b on `A.pcapng`, including all Wireshark filters used, the list of relevant MIDs, and the final answer.

3.2 CQ2)

TODO: Marco: add the complete analysis for CQ2 on `A.pcapng`, including how unsuccessful CON POST and PUT requests were identified and compared per resource.

3.3 CQ3)

TODO: Marco: add the complete analysis for CQ3a and CQ3b on `A.pcapng`, explaining how separate observe notifications for `/dining_room/temperature` were identified and how useless notifications were counted.

3.4 CQ4)

TODO: Marco: add the complete analysis for CQ4 on `A.pcapng`, including the MQTT-SN Wireshark configuration on port 1885 and the counting of messages received by clients from the local broker.

4 Questions on B.pcapng

4.1 CQ5) Last Will messages received through wildcard subscriptions

In MQTT, the *Last Will and Testament* is a message that a client registers when opening the connection with the broker. Conceptually, the client is telling the broker that if the connection is interrupted unexpectedly, without a graceful `DISCONNECT`, the broker must publish a predefined message on a predefined topic. This mechanism is commonly used to notify other clients that a publisher or a device has gone offline unexpectedly.

The question asks how many MQTT subscribers actually receive such a last will message *thanks to a subscription that contains at least one wildcard*. Therefore, the problem cannot be solved by only counting will-enabled clients or by only counting wildcard subscriptions. Instead, it requires combining three conditions: first, a client must define a will topic in the `CONNECT`; second, that will message must actually be published by the broker, which only happens after an abnormal disconnect; third, at least one subscriber must receive it through a matching wildcard-based subscription.

To identify clients that declared a last will, the following filter was used:

```
mqtt.msgtype == 1 and mqtt.conflag.willflag == 1
```

This filter returned four relevant `CONNECT` packets. One of them is directed to the local broker `::1`, while the other three are directed to the broker `5.196.78.28`. The corresponding will topics are:

- `university/department12/room1/temperature` on the local broker;
- `metaverse/room2/floor4` on `5.196.78.28`;
- `hospital/facility3/area3` on `5.196.78.28`;
- `metaverse/room2/room2` on `5.196.78.28`.

At this point, it was necessary to verify whether any subscriber existed on the same broker for those will topics. For the public broker `5.196.78.28`, the filter

```
mqtt.msgtype == 8 and ip.dst == 5.196.78.28
```

showed that no `SUBSCRIBE` requests were directed to that broker. Therefore, even if will messages were configured there, they could not be received by any subscriber, and those three cases can be excluded from the analysis.

The focus then moved to the will topic declared on the local broker. In order to verify whether it was actually published due to an abnormal disconnection, the capture was inspected for graceful `DISCONNECT` packets toward `::1`:

```
mqtt.msgtype == 14 and ipv6.dst == ::1
```

No graceful disconnection corresponding to the client connected at the beginning of the capture was found. This suggested that the connection might have been interrupted abnormally. The hypothesis was confirmed by searching for publishes on the will topic:

```
mqtt.msgtype == 3 and mqtt.topic == "university/department12/room1/temperature"
```

This filter returned four `PUBLISH` packets with that exact topic around time 146 s, which is consistent with the broker publishing the will after detecting the unexpected client disconnection.

The final step was to determine whether any wildcard subscription matched this will topic. Since the will topic belongs to the local broker, the relevant subscriptions were isolated with:

```
mqtt.msgtype == 8 and ipv6.dst == ::1 and mqtt.topic contains "#"
```

The wildcard subscriptions found on the local broker were then checked manually against the four-level topic `university/department12/room1/temperature`. Among them, the only one that matches is:

```
university/#
```

All the other wildcard subscriptions were excluded because at least one topic level did not match the will topic structure or prefix.

Final answer for CQ5: 1.

Only one subscriber receives a last will message through a subscription containing a wildcard. The corresponding client is identified by the Client ID `zmjnxudohrkaegmh`, connected to the local broker `::1`, with subscription `university/#`, which matches the will topic `university/department12`

4.2 CQ6a) Clients that erased a retained value on the public HiveMQ broker

In MQTT, the `retain` flag instructs the broker to store the published message as the last known value for that topic. When a new subscriber later subscribes to the same topic, the broker immediately delivers the retained message, even if no new publication has occurred in the meantime. This is particularly useful in IoT scenarios, where a subscriber often needs the latest available sensor reading as soon as it subscribes.

To remove a retained value from a topic, MQTT uses a specific convention: a client must send a `PUBLISH` with `retain = 1` and an *empty payload* on that same topic. This is not the same as updating the retained value. A publish with `retain = 1` and a non-empty payload simply overwrites the retained message, whereas a publish with `retain = 1` and zero payload explicitly deletes it from the broker state.

The question therefore asks how many distinct MQTT clients performed at least one such retained-message deletion on the public HiveMQ broker. Since HiveMQ was previously identified with the two active IP addresses `18.192.151.104` and `35.158.43.69`, the first step was to isolate all retained publishes directed to those endpoints:

```
mqtt.msgtype == 3 and mqtt.retain == 1 and
(ip.dst == 18.192.151.104 or ip.dst == 35.158.43.69)
```

This filter returns all retained publishes toward HiveMQ, but not all of them erase a previous retained value. To distinguish deletions from updates, the payload field was examined. In Wireshark, after adding `tcp.stream` as an additional column, the packets with empty payload were recognized by the fact that the payload field appeared as `<MISSING>`. Those are exactly the packets corresponding to retained-message deletion.

By analyzing only the retained publishes with empty payload, five distinct TCP streams emerged:

- stream 1
- stream 22
- stream 23
- stream 43
- stream 44

Each TCP stream corresponds to a separate MQTT connection and therefore to a distinct client in this context.

Final answer for CQ6a: 5.

4.3 CQ6b) Of those clients, how many used a Client ID longer than 7 bytes?

The MQTT Client ID is the identifier sent by a client in the `CONNECT` packet when establishing a session with the broker. The broker uses it to distinguish clients, associate persistent session state if required, and manage reconnections. The question asks how many of the five clients identified in CQ6a connected to HiveMQ using a Client ID whose length is strictly greater than 7 bytes.

Starting from the five TCP streams found in CQ6a, the corresponding `CONNECT` packets were isolated with:

```
mqtt.msgtype == 1 and
(tcp.stream == 1 or tcp.stream == 22 or tcp.stream == 23
or tcp.stream == 43 or tcp.stream == 44)
```

For each resulting packet, the MQTT details pane in Wireshark was expanded and the `Client ID` field was read directly together with its byte length. The results are summarized below:

TCP stream	Client ID	Length
1	`pcbtjyj`	7
22	`giuxfijwus`	10
23	`pohujuv`	7
43	`atqeblw`	7
44	`rolsmcwhshlbc`	13

Among these five clients, only two have a Client ID strictly longer than 7 bytes: `giuxfijwus` and `rolsmcwhshlbc`.

Final answer for CQ6b: 2.

4.4 CQ7) Subscribe requests to the local broker with at least two wildcards

MQTT defines two subscription wildcards. The symbol `+` is a single-level wildcard and matches exactly one topic level, while the symbol `#` is a multi-level wildcard and matches all the remaining levels, but it can appear only at the end of the topic. The question asks how many `SUBSCRIBE` requests directed to the local broker specify a topic containing *at least two wildcards*, in any combination. This means that the subscription may contain two `+` symbols, or one `+` plus one `#`, or even more wildcard symbols overall.

To isolate candidate subscriptions, the following filter was applied:

```
mqtt.msgtype == 8 and ipv6.dst == ::1 and
(mqtt.topic contains "+" or mqtt.topic contains "#")
```

This filter returns all wildcard-based subscriptions directed to the local broker `::1`. The topics were then inspected manually one by one, counting the number of wildcard symbols present in each subscription string. The subscriptions containing at least two wildcards are the following:

- `house/+/room1/#`
- `factory/+/#`
- `metaverse/+/+/hydraulic_valve`
- `hospital/fxlugs/+/#`
- `metaverse/facility0/+/+`
- `house/department3/+/+`
- `metaverse/+/room1/#`
- `hospital/+/+/light`
- `university/+/+/deposit`

All the remaining subscribe topics in the filtered output contain at most one wildcard and therefore do not satisfy the condition.

Final answer for CQ7: 9.

5 Question on Both Capture Files

5.1 CQ8)

TODO: Marco: add the complete analysis for CQ8 using both `A.pcapng` and `B.pcapng`. Include the methodology used to count the number of topic layers in MQTT publish messages directed to the local broker, the values for CQ8a and CQ8b, the histogram figure, and optionally a commented script if one was used.

6 Final Answers Summary

For clarity, the final answers obtained in this report section are summarized below.

Question	Answer
CQ5	1
CQ6a	5
CQ6b	2
CQ7	9

TODO: When Marco completes the missing sections, extend this table with CQ1–CQ4 and CQ8.

7 Conclusion

This report presented the analysis carried out on the packet capture `B.pcapng` in order to answer CQ5, CQ6a, CQ6b, and CQ7 of IoT Challenge 2. Before addressing the questions individually, a preliminary network inspection was performed to identify the local broker, the public HiveMQ broker, and the additional public broker appearing in the trace. This step was necessary to build precise Wireshark filters and to avoid mixing traffic belonging to different MQTT endpoints.

The analysis then focused on specific MQTT mechanisms that go beyond basic publish and subscribe operations, namely last will delivery, retained-message deletion, Client ID inspection, and wildcard-based subscriptions. In each case, the answer was obtained by combining protocol semantics with packet-level evidence extracted directly from the capture, rather than by relying on a single filter in isolation.

The resulting answers are the following: exactly one subscriber receives a last will message through a wildcard subscription; five different clients send at least one message to erase a retained value on the HiveMQ broker; among them, two connect with a Client ID longer than seven bytes; and nine subscribe requests directed to the local broker specify a topic containing at least two wildcards.

The remaining parts of the report, concerning `A.pcapng` and the comparison between the two captures for CQ8, can be completed consistently within the same structure.